Directed Functional Verification of an APB–I2C Master Controller

SystemVerilog Testbench and Xcelium/SimVision Simulation

August 2026

Verification Methodology: Directed SystemVerilog Testbench
Simulator: Cadence Xcelium 20.09
Waveform Viewer: SimVision
Final Result: 28 PASS, 0 FAIL

Contents

1 Project Overview **4**

2 Verification Objectives **4**

3 RTL Architecture **5**
3.1 High-Level Block Diagram . . . 5
3.2 APB Wrapper . . . 5

4 Register Map **6**
4.1 Control Register . . . 6
4.2 Command Register . . . 7
4.3 Status Register . . . 7

5 Byte Controller **7**

6 Bit Controller **8**

7 Verification Architecture **9**

8 APB Master Tasks **9**
8.1 `apb_write` . . . 9
8.2 `apb_read` . . . 9

9 Interrupt and Command Completion Handling **10**
9.1 `wait_irq` . . . 10
9.2 `clear_irq` . . . 10

10 Behavioral I2C Slave **10**
10.1 START Detection . . . 10
10.2 STOP Detection . . . 10
10.3 Receiving a Byte . . . 10
10.4 Slave ACK . . . 10
10.5 Sending a Byte . . . 10
10.6 Master NACK Check . . . 10

11 I2C Write Transaction **11**
11.1 Master-Side Sequence . . . 11
11.2 Slave-Side Sequence . . . 11

12 I2C Read Transaction **12**
12.1 Master-Side Sequence . . . 12
12.2 Slave-Side Sequence . . . 13

13 Self-Checking Strategy **13**

14 Directed Test Plan **13**

15 Data Pattern Selection **14**

16 Reset and Recovery Verification **15**

17 Timeout Protection **15**
17.1 Command Timeout . . . 15
17.2 Global Timeout . . . 15

18 Simulation Setup **15**

19 Useful Waveform Signals **16**

20 Expected Waveform: Write **16**

21 Expected Waveform: Read **16**

22 Final Simulation Results **17**

23 What Was Actually Verified **17**

24 What Was Not Claimed as Verified **18**

25 Conclusion **18**

1 Project Overview

This project verifies an APB-programmable I2C master controller using a directed SystemVerilog testbench. The Device Under Test (DUT) exposes an APB slave programming interface and controls the serial I2C bus through SCL and SDA pad-control signals.

The verification environment programs the controller through APB transactions and provides a behavioral I2C slave on the serial side. This allows complete end-to-end checking of the main verified data path:

APB transaction → I2C controller registers → byte controller → bit controller → SCL/SDA bus → behavioral slave

For read operations, the direction is reversed on the data path:

Behavioral slave → SDA → bit controller → byte controller → RX register → APB read

The final directed regression completed successfully with 28 passing checks, zero failures, 18 detected START conditions, and 18 detected STOP conditions.

2 Verification Objectives

The verification was designed to demonstrate the following functionality:

- Correct reset behavior of the APB-visible registers.
- Correct APB write and read access to programmable registers.
- Correct enabling and configuration of the I2C controller.
- Correct generation of an I2C START condition.
- Correct transmission of a 7-bit slave address and R/W bit.
- Correct slave ACK handling during address and write-data phases.
- Correct single-byte I2C write transactions.
- Correct single-byte I2C read transactions.
- Correct master NACK generation for the final byte of a single-byte read.
- Correct STOP generation and return of the bus to idle.
- Correct behavior for representative data patterns.
- Correct behavior for consecutive write and read transactions.
- Correct switching from write operation to read operation.
- Correct operation after reset recovery.
- Correct interrupt-pending polling and interrupt acknowledge handling used by the testbench.

This is a **directed functional verification** environment. It is not claimed to be exhaustive verification of every capability present in the underlying I2C RTL.

3 RTL Architecture

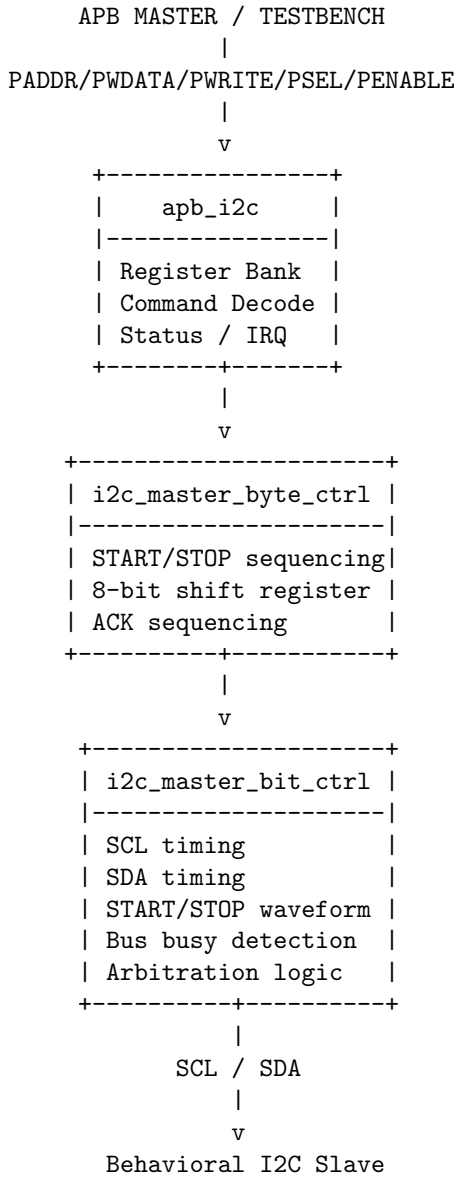
The verified RTL is organized into three principal levels:

1. $\texttt{apb}_i 2c - - - APBwrapper, registerbank, command/statusdecodingandinterruptgeneration.$

1. $\texttt{i2c}_m aster_b yte_c trl - - - byte - levelI2Csequencing.$

1. $\texttt{i2c}_m aster_b it_c trl - - - low - levelSCL/SDAsequencingandbuscontrol.$

3.1 High-Level Block Diagram



3.2 APB Wrapper

The $\texttt{apb}_i 2cmodulehasa12 - bitAPBaddressparameterbydefault.TheAPBinterfaceconsistsof$:

- HCLK --- controller/APB clock.

- HRESETn --- asynchronous active-low reset.
- PADDR --- APB address.
- PWDATA --- APB write data.
- PWRITE --- APB transfer direction.
- PSEL --- APB slave select.
- PENABLE --- APB access-phase indicator.
- PRDATA --- APB read data.
- PREADY --- APB transfer completion.
- PSLVERR --- APB error response.

In this implementation, PREADY is tied high and PSLVERR is tied low. Therefore, the current DUT does not introduce APB wait states or APB slave-error responses.

The wrapper decodes PADDR[5:2] to select the internal register.

4 Register Map

Table 1: APB Register Map

Address	Register	Purpose
0x00	Clock Prescaler	Programs the I2C clock divider
0x04	Control	Core enable and interrupt enable
0x08	RX	Last byte received by the I2C controller
0x0C	Status	ACK, busy, arbitration, TIP and IRQ status
0x10	TX	Byte to be transmitted
0x14	Command	START/STOP/READ/WRITE/ACK/IACK command bits

4.1 Control Register

Only two control bits are used by the wrapper:

Bit	Meaning
7	Core enable
6	Interrupt enable

The directed testbench programs:

```
REG_CTRL = 8'hC0;
```

which enables both the I2C core and its interrupt output.

4.2 Command Register

The command register is decoded as:

Bit	Command
7	START (sta)
6	STOP (sto)
5	READ (rd)
4	WRITE (wr)
3	ACK control (ack)
2:1	Reserved
0	Interrupt acknowledge (iack)

The directed TB uses the following command combinations:

Value	Meaning in the testbench
0x90	START + WRITE
0x50	STOP + WRITE
0x68	STOP + READ + NACK
0x01	Interrupt acknowledge

4.3 Status Register

The wrapper creates the status byte as follows:

Bit	Status
7	Received ACK/NACK status (rxack)
6	I2C bus busy
5	Arbitration lost
4:2	Reserved
1	Transfer in progress (TIP)
0	Interrupt pending flag

The testbench primarily uses bit 0 to determine command completion and bit 7 to check whether the slave acknowledged address/write data.

5 Byte Controller

The byte controller converts high-level byte operations into individual bit-controller commands.
Its principal states are:

ST_IDLE, ST_START, ST_READ, ST_WRITE, ST_ACK, ST_STOP

An 8-bit shift register stores outgoing data or accumulates incoming data. A 3-bit counter tracks the eight serial data bits.

For a write, the byte controller repeatedly issues the bit-level WRITE command. After all eight bits are transmitted, it issues a bit-level READ operation to sample the ACK/NACK returned by the slave.

For a read, it repeatedly issues bit-level READ commands and shifts the sampled SDA values into the receive shift register. After eight bits, it issues a bit-level WRITE operation to transmit the master's ACK/NACK bit.

This distinction is important: an I2C ‘‘read transaction’’ still begins by writing the address byte onto the bus. The R/W bit inside that transmitted address byte is set to 1.

6 Bit Controller

The bit controller converts the simple commands:

START, STOP, WRITE, READ

into actual SCL/SDA transitions.
It also contains logic for:

- I2C clock generation using the prescaler.
- Input synchronization and filtering.
- START detection.
- STOP detection.
- Bus-busy tracking.
- Clock synchronization.
- Slave-wait detection when SCL is externally held low.
- Arbitration-lost detection.

The RTL implements open-drain style control by holding $\mathrm{scl}_o and$

6 Open-Drain Bus Modeling in the Testbench

The directed testbench models SCL and SDA as pulled-up nets:

```
tri1 scl;
tri1 sda;
```

The DUT controls the bus through its output-enable signals:

```
assign scl = (!scl_padoen_o) ? scl_pad_o : 1'bz;
assign sda = (!sda_padoen_o) ? sda_pad_o : 1'bz;
```

The behavioral slave can pull SDA low using:

```
assign sda = slave_sda_low ? 1'b0 : 1'bz;
```

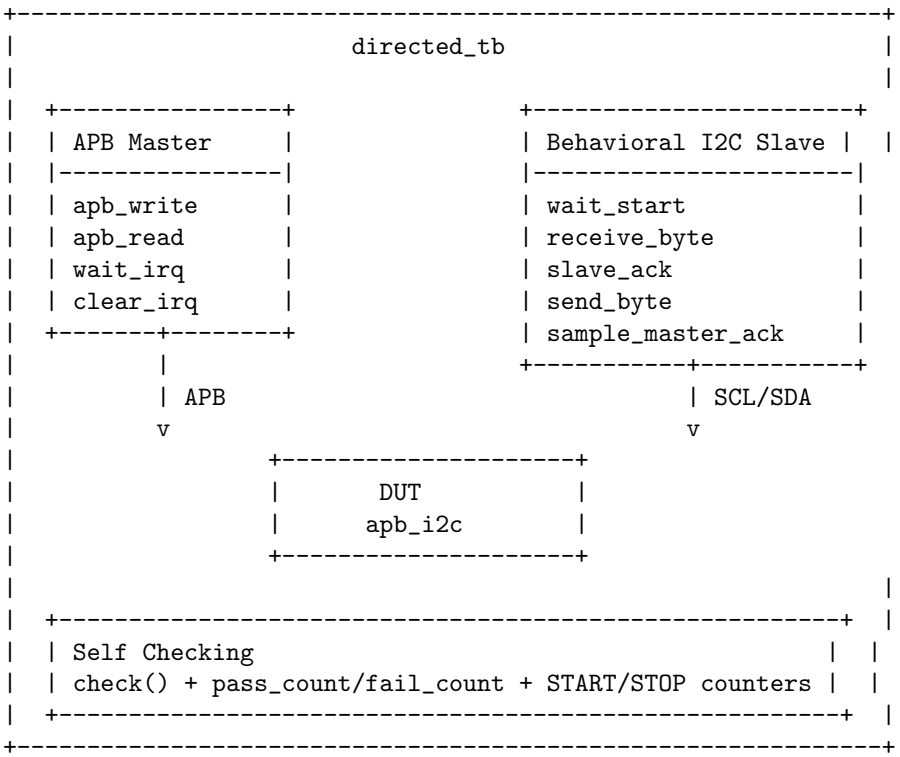
The bus value is fed back to the DUT input pads:

```
assign scl_pad_i = scl;
assign sda_pad_i = sda;
```

This is necessary because I2C devices do not normally drive a logic 1 onto the bus. They either drive the line low or release it, while the pull-up returns the line high.

7 Verification Architecture

The environment is intentionally lightweight and directed rather than class-based/UVM.



8 APB Master Tasks

8.1 apb_write

The APB write task creates the APB setup phase followed by the access phase.
Conceptually:

1. Drive address, data, PWRITE=1, and PSEL=1.
2. Keep PENABLE=0 during setup.
3. Assert PENABLE=1 in the next phase.
4. Check PREADY and PSLVERR.
5. Return APB control signals to idle.

The task drives APB values on the negative edge of HCLK and observes completion after the positive edge, avoiding a same-edge drive/sample race with the DUT.

8.2 apb_read

The APB read task follows the same setup/access sequence, with PWRITE=0. During the access phase it captures PRDATA and checks the APB response.

9 Interrupt and Command Completion Handling

The DUT sets the interrupt-pending flag when a command completes or arbitration is lost. The directed testbench uses this flag as its command-completion indication.

9.1 wait_irq

The task repeatedly reads the status register until:

```
status[0] == 1'b1
```

A timeout prevents the testbench from waiting forever if a transaction fails to complete.

9.2 clear_irq

The testbench writes 0x01 to the command register to acknowledge the interrupt, then confirms that status bit 0 clears.

10 Behavioral I2C Slave

The behavioral slave is not a second synthesizable I2C controller. It is a compact testbench model that produces the serial behavior required to test the master.

10.1 START Detection

A START is detected when SDA falls while SCL is high:

```
@(negedge sda);
if (scl === 1'b1)
    start_count++;
```

10.2 STOP Detection

A STOP is detected when SDA rises while SCL is high.

10.3 Receiving a Byte

The slave samples SDA on each rising edge of SCL, from bit 7 down to bit 0. This verifies the expected MSB-first serial transfer.

10.4 Slave ACK

After receiving a byte, the slave pulls SDA low during the ninth clock. Pulling SDA low represents ACK.

10.5 Sending a Byte

During a master read, the slave places each data bit on SDA while SCL is low so that the value is stable before the next SCL rising edge. The master then samples that bit.

After the eighth data bit, the slave releases SDA so that the master can generate its ACK/NACK bit.

10.6 Master NACK Check

The single-byte read ends after one byte. Therefore, the master is expected to NACK the final received byte before generating STOP. The behavioral slave explicitly checks this condition.

11 I2C Write Transaction

The verified single-byte write flow is:

```
APB: Write {slave_address, 0} into TX
APB: Command = START + WRITE
                 |
                 v
I2C: START
I2C: Address[6:0] + W(0)
I2C: Slave ACK
                 |
                 v
APB: Write data byte into TX
APB: Command = WRITE + STOP
                 |
                 v
I2C: Data[7:0]
I2C: Slave ACK
I2C: STOP
```

11.1 Master-Side Sequence

The master_write_byte task:

1. Writes {slave_addr,1'b0} into TX.
2. Issues 0x90 (START + WRITE).
3. Waits for command completion.
4. Checks status bit 7 for address ACK/NACK.
5. Clears the interrupt.
6. Writes the data byte into TX.
7. Issues 0x50 (WRITE + STOP).
8. Waits for completion.
9. Checks the data ACK/NACK.
10. Clears the interrupt.

11.2 Slave-Side Sequence

In parallel, slave_write_transaction:

1. Waits for START.
2. Receives the address byte.
3. Separates address[7:1] and R/W.
4. ACKs the address.
5. Receives the data byte.
6. ACKs the data.

7. Waits for STOP.

The test passes only when the slave-observed address, direction bit, and data match the expected values.

12 I2C Read Transaction

The verified single-byte read flow is:

```
APB: Write {slave_address, 1} into TX
APB: Command = START + WRITE
                 |
                 v
I2C: START
I2C: Address[6:0] + R(1)
I2C: Slave ACK
                 |
                 v
APB: Command = READ + NACK + STOP
                 |
                 v
I2C: Slave sends Data[7:0]
I2C: Master NACK
I2C: STOP
                 |
                 v
APB: Read RX register
```

The use of START + WRITE for the address phase is intentional. The controller must transmit the address byte, even when the subsequent data phase is a read.

12.1 Master-Side Sequence

The master_read_byte task:

1. Writes {slave_addr,1'b1} into TX.
2. Issues START + WRITE to transmit the address byte.
3. Waits for completion and checks the slave ACK.
4. Clears the interrupt.
5. Issues 0x68: READ + NACK + STOP.
6. Waits for command completion.
7. Reads the RX register.
8. Returns the received byte to the test.
9. Clears the interrupt.

12.2 Slave-Side Sequence

The slave_read_transaction task:

1. Detects START.
2. Receives the address byte.
3. Confirms the R/W bit is 1.
4. ACKs the address.
5. Drives the requested byte onto SDA, MSB first.
6. Samples the master's ninth-bit response.
7. Checks that the master NACKed the final byte.
8. Waits for STOP.

13 Self-Checking Strategy

The environment uses a central check() task.
When a condition is true:

```
pass_count++;
$display("[%0t] PASS : %s", $time, name);
```

Otherwise:

```
fail_count++;
$display("[%0t] FAIL : %s", $time, name);
```

For a write test, the final functional comparison checks:

- observed slave address equals expected address,
- R/W bit equals 0,
- observed slave data equals expected write data.

For a read test, it checks:

- observed slave address equals expected address,
- R/W bit equals 1,
- byte read from the DUT RX register equals the byte driven by the behavioral slave.

This makes the simulation self-checking rather than relying only on manual waveform inspection.

14 Directed Test Plan

Table 2: Directed Test Plan

#	Test	Purpose
1	Bus idle after reset	Verify SCL and SDA are released high after reset.

#	Test	Purpose
2	Prescaler reset value	Verify prescaler resets to zero.
3	Control reset value	Verify control register resets to zero.
4	TX reset value	Verify transmit register resets to zero.
5	Prescaler write/read	Verify APB programming and readback of the clock divider.
6	Control write/read	Verify APB programming and readback of core/interrupt enable.
7	Basic I2C WRITE	Verify complete address + write-data + ACK + STOP path.
8	Basic I2C READ	Verify complete address + read-data + NACK + STOP + RX path.
9	WRITE 0x00	Verify all-zero data pattern.
10	WRITE 0xFF	Verify all-one data pattern.
11	WRITE 0x55	Verify alternating 0101 pattern.
12	WRITE 0xAA	Verify alternating 1010 pattern.
13	READ 0x00	Verify all-zero receive pattern.
14	READ 0xFF	Verify all-one receive pattern.
15	READ 0x55	Verify alternating receive pattern.
16	READ 0xAA	Verify complementary alternating receive pattern.
17	Back-to-back WRITE #1	First consecutive write transaction.
18	Back-to-back WRITE #2	Verify a new write works immediately after a previous write.
19	Back-to-back READ #1	First consecutive read transaction.
20	Back-to-back READ #2	Verify a new read works immediately after a previous read.
21	WRITE before READ	Verify write transaction before direction change.
22	READ after WRITE	Verify controller changes direction correctly.
23	Bus idle after second reset	Verify bus recovery following another reset.
24	WRITE after reset recovery	Verify write functionality after reconfiguration.
25	READ after reset recovery	Verify read functionality after reconfiguration.
26	Final bus idle	Verify no transaction leaves SCL/SDA active.
27	START generated	Confirm at least one valid START was observed.
28	STOP generated	Confirm at least one valid STOP was observed.

15 Data Pattern Selection

The directed patterns 00, FF, 55, and AA were selected because they exercise different serial behavior:

- 00: SDA is driven low for every data bit.
- FF: SDA remains released/high for every data bit.
- 55: repeated low/high transitions.
- AA: complementary repeated high/low transitions.

Together these provide simple but useful directed checking of bit ordering and serial data transitions.

16 Reset and Recovery Verification

The reset task drives HRESETn low for ten clock cycles and then releases it. APB signals and the behavioral slave SDA drive are also returned to inactive values.

Immediately after reset, the TB checks:

- SCL high and SDA high,
- prescaler reset value,
- control register reset value,
- TX register reset value.

Later in the test sequence, reset is asserted again. The controller is reconfigured and both a write and a read are executed. This verifies that normal operation can resume after reset.

17 Timeout Protection

The testbench includes two levels of timeout protection.

17.1 Command Timeout

wait_irq has a bounded polling loop. If the command never completes, the testbench records a failure instead of hanging indefinitely.

17.2 Global Timeout

A global timeout terminates simulation if the complete directed regression fails to finish within the allowed simulation interval:

```
#1_000_000;
$display("[%0t] FAIL : GLOBAL TESTBENCH TIMEOUT", $time);
$finish;
```

The successful final run did not hit this timeout; it terminated normally through the test sequence's own $finish.

18 Simulation Setup

The successful simulation used Cadence Xcelium 20.09 in 64-bit SystemVerilog mode.

The file list contains:

```
-incdir ../apb_i2c-master

../apb_i2c-master/i2c_master_defines.sv
../apb_i2c-master/i2c_master_bit_ctrl.sv
../apb_i2c-master/i2c_master_byte_ctrl.sv
../apb_i2c-master/apb_i2c.sv
./directed_tb.sv
```

The successful GUI invocation was:

```
xrun -64bit -sv -f files.f -access +rwc \
    -timescale 1ns/1ns -gui -l directed.log
```

In SimVision/Xcelium, the test can be allowed to run until the testbench calls $finish:

```
run
```

There is no need to manually estimate a fixed run time for a normal regression.

19 Useful Waveform Signals

For waveform debugging, the most useful signals are:

- APB: HCLK, HRESETn, PADDR, PWDATA, PRDATA, PWRITE, PSEL, PENABLE, PREADY.
- I2C: scl, sda.
- Interrupt: interrupt_o.
- Slave observation: slave_seen_addr, slave_seen_rw, slave_seen_data, slave_tx_data.
- Result tracking: pass_count, fail_count, start_count, stop_count.

For protocol interpretation, the resolved scl and sda nets are more useful than looking only at the pad output values, because the physical bus state depends on output-enable/release behavior and the slave's drive.

20 Expected Waveform: Write

A valid single-byte write should appear conceptually as:

```
SDA:  START  A6 A5 A4 A3 A2 A1 A0 0  ACK  D7...D0 ACK  STOP
SCL:         _|_|_|_|_|_|_|_|_|_| ...          _|_
             address + write          data
```

Important observations:

- START: SDA falls while SCL is high.
- Address is transferred MSB first.
- R/W bit is 0.
- Slave pulls SDA low during ACK clocks.
- Data is transferred MSB first.
- STOP: SDA rises while SCL is high.

21 Expected Waveform: Read

A valid single-byte read should appear conceptually as:

```
SDA: START A6 A5 A4 A3 A2 A1 A0 1 ACK D7...D0 NACK STOP
SCL:       _|_|_|_|_|_|_|_|_|_| ...       _|_
           address + read          slave data
```

During the data byte, the DUT releases SDA and the behavioral slave controls the data value. After the eighth bit, the slave releases SDA and the master leaves SDA high to NACK the final byte, then generates STOP.

22 Final Simulation Results

The final Xcelium run reported:

```
================================================
            DIRECTED TEST REPORT
================================================
PASS        = 28
FAIL        = 0
START COUNT = 18
STOP COUNT  = 18
================================================
        APB-I2C VERIFICATION PASSED
================================================
```

Simulation completed normally at:

232665 ns

The individual log entries confirmed successful basic read/write operations, write and read data patterns, consecutive operations, direction switching, reset recovery, final bus-idle checking, and START/STOP detection.

Table 3: Final Regression Summary

Metric	Result
Passing checks	28
Failing checks	0
START conditions detected	18
STOP conditions detected	18
Simulation end time	232665 ns
Overall result	PASS

The equal START and STOP counts provide an additional sanity check that each directed transaction completed with a bus-termination condition.

23 What Was Actually Verified

The final directed regression provides evidence for the following:

- APB register reset behavior for the registers explicitly checked.
- APB write/read programming of the prescaler and control registers.
- Core enable and interrupt-enable configuration used by the test.
- Single-byte 7-bit-address I2C master writes.
- Single-byte 7-bit-address I2C master reads.
- Slave ACK reception during address and write-data phases.
- Master final-byte NACK during the directed single-byte read.
- START and STOP generation.

- MSB-first serial transfer through the byte-oriented tasks.
- Multiple representative read/write data patterns.
- Consecutive writes.
- Consecutive reads.
- Write-to-read direction switching.
- Operation after reset and reconfiguration.
- Bus return to idle after transfers.
- Command completion/interrupt-pending polling used by the TB.

24 What Was Not Claimed as Verified

The underlying RTL contains logic related to capabilities beyond the current directed regression. The present testbench does not provide sufficient evidence to claim exhaustive verification of those features.

The current directed test does not explicitly verify:

- Repeated START transactions.
- Multi-byte sequential read/write transfers in one transaction.
- Slave-generated NACK error scenarios.
- Clock stretching using a slave that actively holds SCL low.
- Multi-master arbitration behavior.
- Arbitration-lost recovery.
- 10-bit I2C addressing.
- Invalid/reserved command combinations.
- APB wait-state handling, because this DUT ties PREADY high.
- APB error-response behavior, because this DUT ties PSLVERR low.
- Exhaustive timing/specification compliance.
- Functional or code coverage closure.

This distinction is important in an interview: a feature existing in RTL is not the same as that feature having been verified by this particular testbench.

25 Conclusion

A directed SystemVerilog verification environment was developed for the APB--I2C master controller. The testbench drives the DUT through its APB programming interface while a behavioral I2C slave interacts with the serial bus.

The verification exercises the complete single-byte write and read paths rather than checking only isolated register behavior. It verifies address and direction transmission, ACK behavior, serial data transfer, RX data, START/STOP generation, representative data patterns, consecutive operations, direction switching, and reset recovery.

The final Xcelium simulation completed normally with:

28 PASS, 0 FAIL, 18 START, 18 STOP

This establishes a clean directed-verification baseline for the verified functionality and provides a foundation for future negative testing, protocol assertions, randomized stimulus, and coverage-driven verification.